

Announcements

- Please make sure you are not using any other device in weekly quizzes.
- Assignment in today's class based on Maven
- **Before the TOTT demo, please have ready:**
 - JDK 21 installed → Check *java --version*
 - VS Code installed
 - **Extension Pack for Java** installed in VS Code
 - Maven 3.9.x installed → Check *mvn -version*
 - Links on slide 2 on the class website

Install JDK, Maven

How to Install JDK

1. Browse to the [AdoptOpenJDK](https://adoptopenjdk.org/) website.
2. Choose your **Operating System**
3. Package type is **JDK**
4. **Choose v21**
5. Click the **Latest release** download button to download the package.

How to Install and setup VSCode

1. Browse to <https://code.visualstudio.com/download>. Select Download based on your device type
2. Once installed, open VS Code and install the **Extension Pack for Java** from the Extensions Marketplace. This pack includes the essential tools for Java development in VS Code.

Install Maven

1. Download/install [Apache Maven](https://maven.apache.org/)
2. Ensure Maven is added to your system **PATH**
3. Verify: mvn -version.
4. Confirm the output shows **Maven 3.9.x and Java 21**

CS3300 Introduction to Software Engineering

Lecture 05: Tools of the Trade #2

VS Code IDE, Junit Testing, Maven

Dr. Nimisha Roy ▶ nroy9@gatech.edu

Where do these tools fit?

A Development Workflow

WRITE CODE



BUILD



TEST

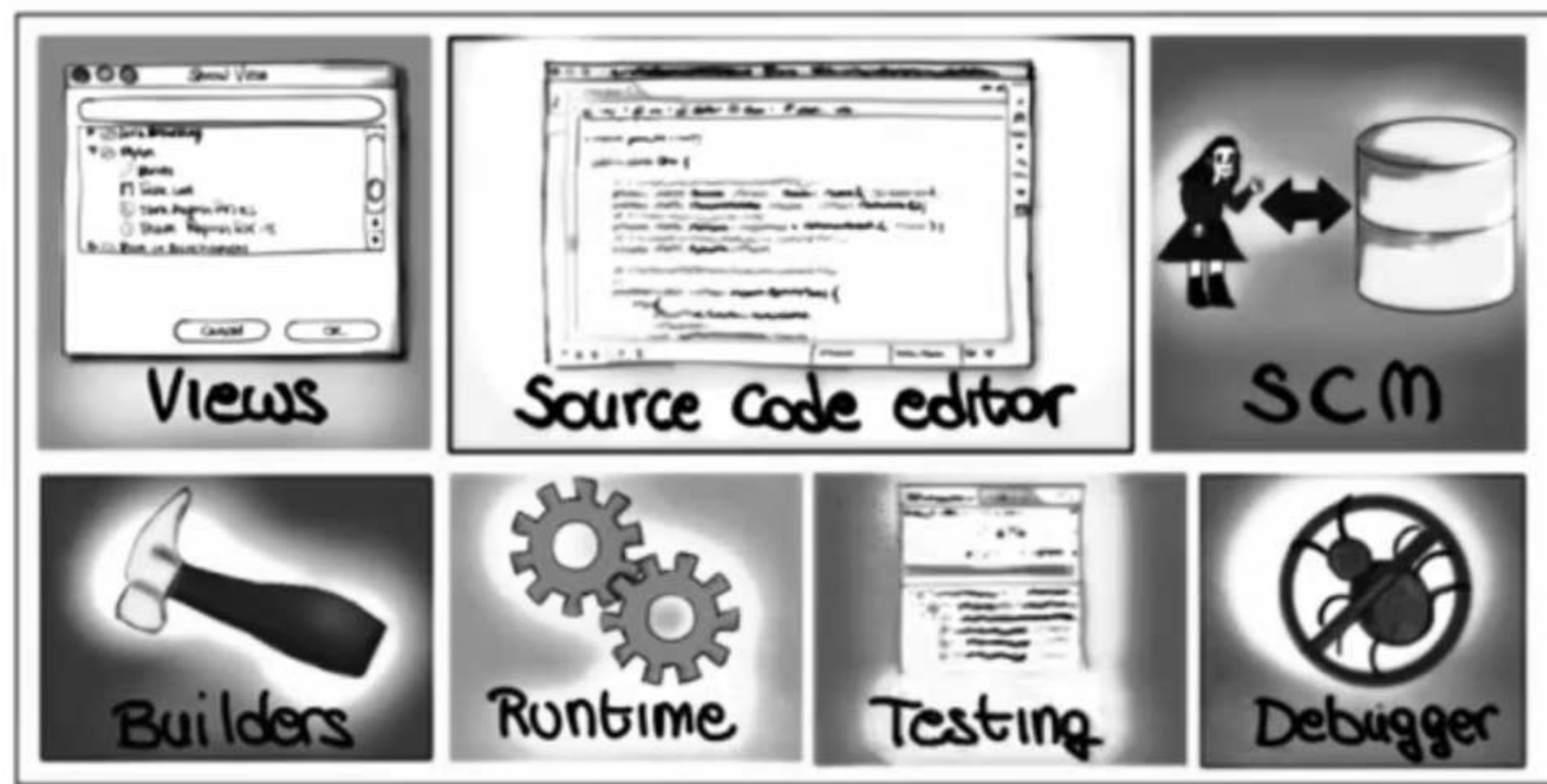


DEBUG / FIX



Tool	Role
VS Code	Write, navigate, run & debug
Maven	Build + dependencies
JUnit	Automated testing

What is an IDE?



- Integrated Development Environments (IDEs) are software applications that support developers in many of their everyday tasks, such as writing, compiling, and debugging code.
- Some IDEs are designed to support only one programming language such as Java, while others can be used for various languages
- Most popular Java IDEs: VSCode, IntelliJ IDEA, Eclipse, Netbeans

What is an IDE?- VSCode

- VS Code is a lightweight, open-source IDE developed by Microsoft.
- It is highly versatile and supports multiple languages, including Java, Python, JavaScript, and C++.
- Plug-ins provide additional functionality to VS Code, allowing developers to customize the IDE to suit their specific needs. For example, the "Extension Pack for Java" adds Java support, and there are many other plugins for various programming languages and tools, such as Git, Docker, and more.
- VS Code is available on all major operating systems: Mac, Windows, and Linux.

Maven: Why do we need it?

Imagine managing a Java project manually...

You need to:

- Find libraries → Download JARs → Configure dependencies → Compile → Test → Package

Maven automates this workflow

mvn test

Maven provides:

- **Dependency management**
- **Standard project structure**
- **Repeatable builds**
- **Automated compilation and testing**
- **Packaging**

Maven

- Powerful build management tool that can be used for building and managing any Java based project.
 - Easy building of project
 - Generate source code, generate documentation from source code (log document, dependency list, unit test reports etc)
 - Getting right dependencies for project, Compiling source code
 - Packaging compiled codes into JAR, WAR files without scripting
 - Installing packaged code in local, server or central repositories
- Based on POM (Project Object Model)
 - POM files are XML files that contain information related to the project and configuration information such as dependencies, source directory, plugin, goals etc. used by Maven to build the project.

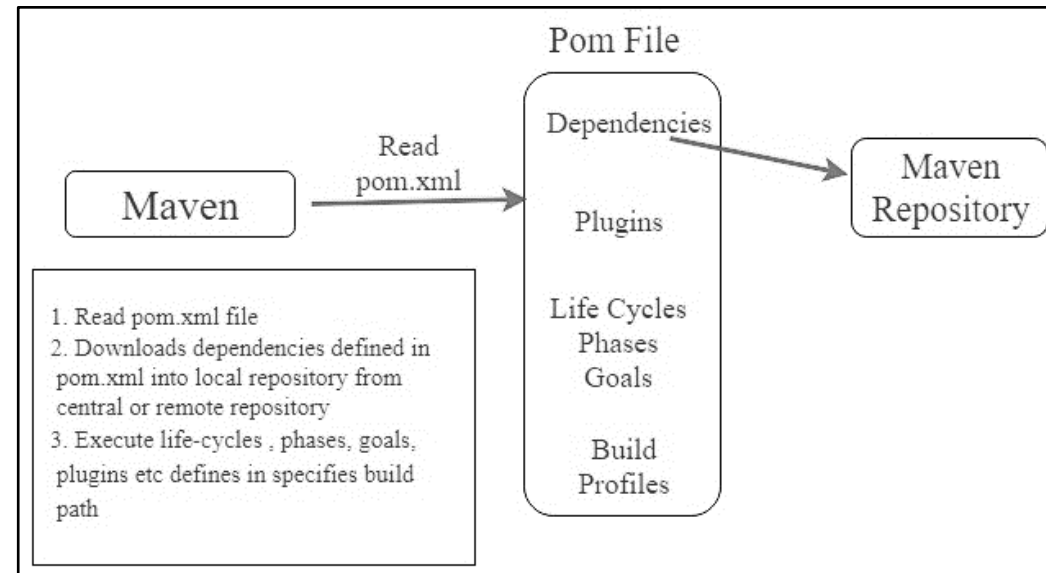


Image courtesy: <https://www.geeksforgeeks.org/introduction-apache-maven-build-automation-tool-java-projects/>

Maven and pom.xml

Standard Project Anatomy

```
my-project/  
|  
├─ pom.xml  
|  
└─ src/  
    ├─ main/  
    │   └─ java/  
    │       └─ Application.java  
    └─ test/  
        └─ java/  
            └─ ApplicationTest.java
```

pom.xml describes the project and its dependencies.

```
<dependency>  
    <groupId>org.junit.jupiter</groupId>  
    <artifactId>junit-jupiter</artifactId>  
    <version>...</version>  
    <scope>test</scope>  
</dependency>
```

```
mvn compile  
mvn test  
mvn package  
mvn clean
```

JUnit Testing

Unit Test -A small automated test that verifies a specific behavior.

@Test

```
void addsTwoNumbers() {  
    assertEquals(5, calculator.add(2, 3));  
}
```

Why automate tests?

- Catch defects early
- Prevent **regressions**
- Make changes with more confidence
- Provide repeatable verification
- Support Test-Driven Development (TDD)

Regression - Previously working behavior breaks after a change.

JUnit Best Practices

- **Arrange** → establish initial state
Act → execute the behavior
Assert → verify the result
- Keep test cases separate from source code (src/test)
- Method names need to be specific
- Keeping tests simple and focused on one feature or expected result
- Making sure to test for edge cases and not only “perfect scenarios”
- Use appropriate assertions to verify what's expected and what the function returns (actual)
- Tests are repeatable and reliable

1. *assertEquals*(expected, actual): Verifies that the expected and the actual values are equal.

2. *assertNotEquals*(first, second): Verifies that two values are not equal.

3. *assertTrue*(condition): Verifies that the condition is true.

4. *assertFalse*(condition): Verifies that the condition is false.

5. *assertNull*(object): Verifies that the object is null.

6. *assertNotNull*(object): Verifies that the object is not null.

What Should We Test?

Suppose an online store calculates shipping:

```
public double calculateShipping(double total, boolean member) {  
    if (total >= 50) {  
        return 0.0;  
    } else if (member) {  
        return 4.99;  
    } else {  
        return 8.99;  
    }  
}
```

What cases should we test?

From Local Testing to Automation

On your computer

Write code



mvn test



JUnit tests run

Automated workflow

Push code



GitHub Actions



mvn test



JUnit tests run

Same tests. Different trigger.

Demo – Run unit tests with Maven in VSCode

We will:

- Verify Java + Maven
- Create a Maven project
- Explore the project structure
- Examine *pom.xml*
- Write Java methods
- Write JUnit tests
- Run tests in VS Code
- Run *mvn test*
- Intentionally fail a test
- Debug and fix the problem

Demo- Debugging

When a test fails:

Failing Test → Inspect State → Step
Through Code → Identify Cause → Fix →
Re-run Tests

Java Debugger

Included with the **Extension Pack for Java**
Set breakpoints directly in the editor
Inspect variables while execution is paused

Type of breakpoint	When does it stop?	Example
Regular	When a line is reached	Stop at shipping condition
Conditional	Line reached and condition is true	orderTotal == 50.0
Data	A supported field changes	Watch a field change
Logpoint	Doesn't stop — logs information	Print values to Debug Console

What did Maven do for us? What would we have needed to do without Maven?

With Maven	Without a build tool
Declare JUnit in pom.xml	Find/download the appropriate JUnit JARs
mvn compile	Invoke javac and configure compilation
Dependencies resolved automatically	Manage dependencies and compatible versions
Classpath configured	Manually construct the classpath
mvn test	Compile tests and invoke the JUnit runner
Standard src/main / src/test structure	Establish/manage your own conventions
mvn package	Manually create/package build artifacts
Reproducible project configuration	Give other developers setup/build instructions

Common Demo Troubleshooting

Symptom	Likely cause / action
"There is no POM in this directory"	You are not in the Maven project directory. cd to the folder containing pom.xml.
"Source option X is no longer supported"	Generated POM targets an obsolete Java version. Use maven.compiler.release=21.
Dependency org.junit.jupiter:junit... not found	Wrong artifactId. JUnit 5 dependency is org.junit.jupiter:junit-jupiter:5.11.2.
package org.junit does not exist after switching to JUnit 5	A generated JUnit 4 test/import remains. Delete/convert AppTest.java and use org.junit.jupiter imports.
VS Code red underline but mvn test succeeds	Refresh Java language server / reload window before changing correct code.
Cannot find debug action	Verify Microsoft Debugger for Java and Test Runner for Java are installed/enabled; resolve conflicting Java extensions.

Assignment